NEXT ENGINEER · _SHEERAT_

Data Compression

BCS064

One-day revision notes: definitions, solved examples, code and important questions

Program: B.Tech | **Branches:** CSE, CS, IT | **Semester:** 6 | **Credits:** 3 | **L-T-P:** 3-0-0

Scheme: AKTU B.Tech 3rd year, Departmental Elective, 2024-25 onwards

UNIT 1	UNIT 2	UNIT 3	UNIT 4	UNIT 5
Compression Techniques, Modeling and Coding	Huffman, Golomb, Rice and Tunstall Coding	Arithmetic Coding, Dictionary Techniques and Context-Based Compression	Distortion Criteria and Scalar Quantization	Vector Quantization

Original revision notes written for the AKTU syllabus. Every unit has the same parts:

1. **2-mark definitions**: short answers you can write word for word in Section A.
2. **Core concepts**: algorithms, formulas and comparison tables.
3. **Solved examples**: the numericals that appear every year (entropy, Huffman and minimum variance Huffman, Golomb, Rice, Tunstall, arithmetic coding, LZ77, LZ78, LZW, BWT and move-to-front, quantizer design, LBG).
4. **Formulas to remember**.
5. **Important questions**: the long questions to prepare, with what your answer must contain.

How to use these notes the day before the paper

- **Morning (3 hours):** Unit 1 (measures, entropy, models, uniquely decodable and prefix codes, Kraft inequality) and Unit 2 (Huffman, minimum variance Huffman, adaptive Huffman, Golomb, Rice, Tunstall).
- **Afternoon (3 hours):** Unit 3 (arithmetic coding with tag and binary code, JBIG, LZ77, LZ78, LZW, GIF, compress, V.42bis, PPM, BWT, MTF, CALIC, JPEG-LS, facsimile, DMC).
- **Evening (2 hours):** Unit 4 (distortion measures, uniform, adaptive and non-uniform quantizers) and Unit 5 (vector quantization, LBG, tree-structured and structured VQ). Then the "Last-hour checklist".
- **In the exam:** for every coding question show the table (symbol, probability, code, length), the average length, the entropy and the efficiency. For dictionary methods show the dictionary as it grows and the output step by step.

AKTU paper pattern (theory, 70 marks, 3 hours): Section A has 7 short questions of 2 marks, all compulsory (14). Section B has 5 questions of 7 marks, attempt any 3 (21). Section C has 5 questions (one per unit), each with two parts, attempt one part of each, 7 marks each (35). Every unit gets its own Section C question, so do not skip any unit.

UNIT 1 Compression Techniques, Modeling and Coding

✍️ 2-mark definitions

- **Data compression:** representing information with fewer bits than the original representation, to save storage and transmission bandwidth.
- **Lossless compression:** the original data can be reconstructed exactly from the compressed data (text, executable files, medical images). Examples: Huffman, arithmetic coding, LZW, PNG, ZIP.
- **Lossy compression:** the reconstruction differs from the original, but the difference is acceptable; gives much higher compression (images, audio, video). Examples: JPEG, MP3, MPEG.
- **Compression ratio:** size of original data / size of compressed data (for example 4:1).
- **Rate:** average number of bits per sample or symbol (bits/pixel, bits/symbol).
- **Distortion:** the difference between the original and the reconstructed data in lossy compression (measured by MSE, SNR, PSNR).
- **Fidelity / quality:** how close the reconstruction is to the original.
- **Modeling:** describing the structure (redundancy) in the data, for example by probabilities or prediction.
- **Coding:** assigning binary codewords to the symbols or to the residual from the model.
- **Self-information:** $i(A) = -\log_2 P(A)$ bits; rare events carry more information.
- **Entropy:** the average self-information of a source: $H = -\Sigma P(a_i) \log_2 P(a_i)$ bits/symbol. It is the lower bound on the average code length for lossless coding of a memoryless source.
- **Uniquely decodable code:** any sequence of codewords can be decoded in only one way.
- **Prefix code (instantaneous code):** no codeword is a prefix of another; it can be decoded as soon as each codeword ends.
- **Markov model:** a model where the probability of the next symbol depends on the previous k symbols (k-th order Markov).

💡 Core concepts

Lossless vs lossy

Lossless	Lossy
Exact reconstruction	Approximate reconstruction
Lower compression ratio (typically 2:1 to 4:1)	Higher ratio (10:1 to 100:1 or more)
Text, programs, databases, medical and legal images	Photos, audio, video, streaming
Huffman, arithmetic, LZ family, run-length, PNG, FLAC	JPEG, MP3, AAC, MPEG, H.264
Measured by ratio and rate	Measured by rate and distortion (rate-distortion trade-off)

Performance measures: compression ratio, rate (bits/sample), distortion (MSE, SNR, PSNR), speed and complexity of encoding and decoding, memory used, robustness to errors.

Modeling and coding: compression = model (captures structure) + coder (represents what remains). Example: data 9, 11, 11, 11, 14, 13, 15, 17, 16, 17, 20, 21 follow roughly $\hat{x}n = n + 8$ (n = 1, 2, ...); residuals 0, 1, 0, −1, 1, −1, 0, 1, −1, −1, 1, 1 need only 2 bits each instead of 5. Another model: send the first value and then differences (delta coding).

Information theory basics

- Self-information $i(A) = -\log_2 P(A)$; for independent A and B, $i(AB) = i(A) + i(B)$.
- Entropy of a source $H = -\Sigma P(ai) \log_2 P(ai)$; maximum $\log_2 n$ when all n symbols are equally likely.
- First-order entropy treats symbols as independent; using larger blocks or context (conditional entropy) gives lower entropy when symbols are correlated.
- **Shannon's source coding theorem:** the average length of any uniquely decodable code satisfies $\bar{l} \geq H$; a code exists with $H \leq \bar{l} < H + 1$. Coding blocks of n symbols gives $H \leq \bar{l} < H + 1/n$.

Models

1. **Physical model:** based on knowledge of how the data is produced (for example a speech production model of the vocal tract).
2. **Probability model:** each symbol has a fixed probability, independent of others (ignorance model when all are equal).
3. **Markov model:** probability depends on the previous k symbols; for binary images a two-state Markov model (white and black pixels) captures runs. Entropy of a Markov source $H = \Sigma P(Si) H(Si)$, where H(Si) is the entropy of the state's outgoing probabilities.
4. **Composite source model:** several sources, with a switch selecting which one is active at a time (for example text and images in one document).

Coding: uniquely decodable and prefix codes

- **Kraft-McMillan inequality:** a uniquely decodable code with lengths l1, ..., ln must satisfy $\Sigma\ 2^{(-li)} \leq 1$. Conversely, if the lengths satisfy it, a prefix code with those lengths exists. So nothing is lost by restricting to prefix codes.
- **Test for unique decodability (Sardinas-Patterson):** list all codewords; whenever one codeword is a prefix of another, add the "dangling suffix" to the list; repeat with new suffixes. If a dangling suffix is itself a codeword, the code is **not** uniquely decodable; if no new suffixes appear, it **is**.

🧮 Solved examples

Example 1 (entropy). Source with P = {0.5, 0.25, 0.125, 0.125}. H = 0.5 × 1 + 0.25 × 2 + 0.125 × 3 + 0.125 × 3 = **1.75 bits/symbol**. A fixed-length code needs 2 bits; the prefix code {0, 10, 110, 111} has average length exactly 1.75 bits (100% efficient).

Example 2 (unique decodability).

- Code {0, 01, 11}: 0 is a prefix of 01 → dangling suffix 1. 1 is not a codeword and is not a prefix of any codeword, and no codeword is a prefix of it, so no new suffixes → **uniquely decodable** (but not a prefix code, so decoding may need look-ahead).
- Code {0, 01, 10}: 0 is a prefix of 01 → suffix 1; 1 is a prefix of 10 → suffix 0, which is a codeword → **not uniquely decodable** (010 = 0·10 or 01·0).

Example 3 (Kraft). Lengths {1, 2, 3, 3}: 1/2 + 1/4 + 1/8 + 1/8 = 1 ≤ 1, so a prefix code exists ({0, 10, 110, 111}). Lengths {1, 1, 2}: 1/2 + 1/2 + 1/4 = 1.25 > 1, so no uniquely decodable code exists.

Example 4 (compression ratio and rate). A 256 × 256 image with 8 bits per pixel = 65,536 bytes. Compressed to 16,384 bytes: ratio = **4:1**; rate = 8/4 = **2 bits/pixel**.

Formulas to remember

- $i(A) = -\log_2 P(A)$; $H = -\Sigma P \log_2 P$.
- $H \le \bar{l} < H + 1$; efficiency = $H/\bar{l}$; redundancy = $\bar{l} - H$.
- Kraft: $\Sigma\ 2^{(-l_i)} \le 1$.
- Compression ratio = original/compressed; rate = bits per symbol.

⭐ Important questions

1. Differentiate lossless and lossy compression. Explain the measures of performance.
2. Explain modeling and coding with an example.
3. Define self-information and entropy. State Shannon's source coding theorem.
4. Explain physical, probability, Markov and composite source models.
5. What is a uniquely decodable code? Explain the test for unique decodability and the Kraft-McMillan inequality. What are prefix codes?

NEXT ENGINEER _sheerat_

UNIT 2 Huffman, Golomb, Rice and Tunstall Coding

✍️ 2-mark definitions

- **Huffman code:** an optimal prefix code (minimum average length among symbol-by-symbol codes) built by repeatedly merging the two least probable symbols.
- **Minimum variance Huffman code:** a Huffman code built by placing merged nodes as high as possible in the sorted list, giving codeword lengths with the least variance (useful for buffer design at fixed-rate channels).
- **Adaptive (dynamic) Huffman coding:** a one-pass method where the encoder and decoder update the same Huffman tree as symbols are processed, so no statistics need to be sent in advance.
- **Sibling property:** in a Huffman tree, nodes can be listed in non-increasing order of weight so that each node is adjacent to its sibling; adaptive Huffman maintains this.
- **NYT (Not Yet Transmitted) node:** the special node in adaptive Huffman used to signal a symbol seen for the first time.
- **Golomb code:** a code for non-negative integers with a parameter m, optimal for geometrically distributed data: unary code of the quotient followed by a truncated binary code of the remainder.
- **Rice code:** a Golomb code with m = 2^k, so the remainder is simply k bits; very fast (used in lossless audio and in space data standards).
- **Tunstall code:** a variable-to-fixed length code: variable-length sequences of source symbols are mapped to fixed-length codewords.
- **Extended Huffman code:** Huffman coding applied to blocks of symbols to get closer to the entropy for skewed sources.

💡 Core concepts

Huffman algorithm

1. Sort symbols by probability.
2. Merge the two least probable into a new node with the sum of probabilities.
3. Repeat until one node remains.
4. Assign 0 and 1 to the two branches at each merge; read codewords from the root to each leaf. - Properties: symbols with higher probability get shorter codes; the two least probable symbols have codewords of the same length that differ only in the last bit. $H \le \bar{l} < H + 1$ (more precisely $< H + p_{max} + 0.086$).

Adaptive Huffman (FGK/Vitter)

- **Start:** a tree with only the NYT node (weight 0).
- **Encoding a symbol:** if it has been seen, send its current code from the tree; if it is new, send the NYT code followed by a fixed code for the symbol (for example its index).

- **Update procedure:** for a new symbol, NYT gives birth to a new NYT and a leaf for the symbol. Then, going from the symbol's leaf up to the root: if the node is not the highest-numbered node in its weight block, swap it with that node (unless it is its parent); increment its weight; move to the parent.
- **Decoding:** the decoder follows the bits down the same tree; at NYT it reads the fixed code to identify the new symbol; then it performs the same update so both trees stay identical.

Golomb coding (parameter m)

- For integer n: q = ⌊n/m⌋, r = n − qm.
- Codeword = unary(q) (q ones followed by a zero) + truncated binary code of r: let b = ⌈log2 m⌉; the first 2^b − m values of r use b − 1 bits; the rest use b bits for r + 2^b − m.
- Best when P(n) = (1 − p) p^n (geometric), choosing m ≈ ⌈−1/log2 p⌉.

Rice coding: m = 2^k: unary(q) followed by the k least significant bits of n. Used in FLAC, Shorten, the CCSDS lossless standard and JPEG-LS (adaptive k).

Tunstall coding (n-bit codewords, alphabet size N)

1. Start with the N single symbols.
2. Remove the most probable entry and add its N extensions (entry followed by each symbol). Each step adds N − 1 entries.
3. Repeat while the number of entries + (N − 1) ≤ 2^n.
4. Assign n-bit codes to the entries. - Advantage: fixed-length codewords mean an error affects only one codeword (good for error resilience and simple decoding).

Applications of Huffman coding

- **Lossless image compression:** code the pixel values directly (poor) or code the differences between neighbouring pixels (much better, because differences cluster around zero).
- **Text compression:** characters have very uneven probabilities (e, t, a frequent; q, z rare); used in ZIP/DEFLATE together with LZ77.
- **Audio compression:** code differences between successive samples; used in MP3 and AAC for coding quantized values.
- Also in JPEG (baseline uses Huffman tables for DCT coefficients) and fax.

🧮 Solved examples

Example 1 (Huffman and minimum variance Huffman). Symbols a1 = 0.2, a2 = 0.4, a3 = 0.2, a4 = 0.1, a5 = 0.1.

- Entropy H = **2.122 bits/symbol**.
- Huffman (merged node placed low): a2 = 1, a1 = 01, a3 = 000, a4 = 0010, a5 = 0011. Lengths 1, 2, 3, 4, 4.
- Minimum variance (merged node placed as high as possible): a1 = 10, a2 = 00, a3 = 11, a4 = 010, a5 = 011. Lengths 2, 2, 2, 3, 3.

Code	Average length	Variance of length	Efficiency
Huffman (first)	0.4×1 + 0.2×2 + 0.2×3 + 0.1×4 + 0.1×4 = **2.2**	**1.36**	2.122/2.2 = **96.5%**
Minimum variance	0.2×2 + 0.4×2 + 0.2×2 + 0.1×3 + 0.1×3 = **2.2**	**0.16**	**96.5%**

Both have the same average length; the minimum variance code is better for transmission at a fixed rate because the bit rate varies less (smaller buffer).

Example 2 (Golomb code, m = 5). b = ⌈log2 5⌉ = 3; $2^3 - 5 = 3$, so r = 0, 1, 2 use 2 bits (00, 01, 10) and r = 3, 4 use 3 bits for r + 3 (110, 111).

n	q	r	Codeword
0	0	0	0 00
3	0	3	0 110
4	0	4	0 111
7	1	2	10 10
9	1	4	10 111
10	2	0	110 00

Example 3 (Rice code, k = 2, so m = 4). n = 9: q = 2 → 110; r = 1 → 01. Codeword = **11001**.

Example 4 (Tunstall, 3-bit codes). Alphabet A (0.6), B (0.3), C (0.1); N = 3, $2^3 = 8$.

- Start: A, B, C (3 entries).
- Expand A (0.6): AA (0.36), AB (0.18), AC (0.06), B, C (5 entries).
- Expand AA (0.36): AAA, AAB, AAC, AB, AC, B, C (7 entries).
- Another expansion would give 9 > 8, so stop.
- Codes: AAA 000, AAB 001, AAC 010, AB 011, AC 100, B 101, C 110 (code 111 unused).

Example 5 (adaptive Huffman intuition). Encoding "aab" with the fixed code for new letters a = 00000, b = 00001: first 'a' → NYT code (empty at start) + 00000; tree now has NYT and a(1). Second 'a' → code for a (1). 'b' → NYT code (0) + 00001. After each symbol both sides update the tree identically.

Formulas to remember

- Average length $\bar{l} = \Sigma P(a_i) l_i$; variance = $\Sigma P(a_i)(l_i - \bar{l})^2$; efficiency = $H/\bar{l}$.
- Golomb: q = ⌊n/m⌋, r = n mod m; b = ⌈log2 m⌉; first 2^b − m remainders in b − 1 bits.
- Rice: m = 2^k; codeword = unary(q) + k bits of r.
- Tunstall: entries after each expansion increase by N − 1; stop when next exceeds 2^n.

⭐ Important questions

1. Explain the Huffman coding algorithm and prove/argue its optimality properties. Construct Huffman and minimum variance Huffman codes and compare them.
2. Explain adaptive Huffman coding: update procedure, encoding and decoding.
3. Explain Golomb and Rice codes with examples.
4. Explain Tunstall coding with an example.
5. Explain applications of Huffman coding in lossless image, text and audio compression.

UNIT 3 Arithmetic Coding, Dictionary Techniques and Context-Based Compression

✍️ 2-mark definitions

- **Arithmetic coding:** a method that represents an entire sequence of symbols by a single number (tag) in the interval [0, 1), by repeatedly narrowing the interval according to symbol probabilities.
- **Tag:** a number in the final interval that identifies the sequence (often the midpoint).
- **Cumulative distribution function (CDF) F(x):** the sum of probabilities of all symbols up to and including x; the interval for symbol x is [F(x − 1), F(x)).
- **Dictionary coding:** replacing repeated strings in the data with references (indices) to a dictionary of strings.
- **Static dictionary:** a fixed dictionary built in advance (for example digram coding).
- **Adaptive dictionary:** built during encoding from the data itself (LZ77, LZ78, LZW).
- **LZ77:** sliding-window method that encodes a string as a triple ⟨offset, length, next symbol⟩ pointing back into the recently seen text.
- **LZ78:** encodes input as pairs ⟨index, next symbol⟩, where the index refers to a growing dictionary of phrases.
- **LZW:** a variant of LZ78 that starts with all single symbols in the dictionary and sends only indices.
- **PPM (Prediction by Partial Matching):** a context-modelling method that predicts the next symbol using the longest previous context in which it has appeared, with an escape mechanism to shorter contexts.
- **BWT (Burrows-Wheeler Transform):** a reversible transform that sorts all rotations of a block and outputs the last column, grouping similar symbols together for easier compression.
- **Move-to-front (MTF) coding:** replaces each symbol by its position in a list and moves it to the front, so repeated symbols become small numbers (zeros).
- **JBIG / JBIG2:** standards for compression of bi-level (black and white) images using context-based arithmetic coding; JBIG2 adds pattern matching and text/halftone regions.
- **CALIC and JPEG-LS:** predictive lossless image coders using context modelling (JPEG-LS uses the LOCO-I algorithm with Golomb-Rice coding).

💡 Core concepts

Arithmetic coding: generating the tag

- Start with $l(0) = 0$, $u(0) = 1$.
- For each symbol x_k: $l(k) = l(k-1) + (u(k-1) - l(k-1))\, F(x_k - 1)$; $u(k) = l(k-1) + (u(k-1) - l(k-1))\, F(x_k)$.
- Tag $= (l(n) + u(n))/2$.
- **Generating a binary code:** truncate the binary expansion of the tag to $l(x) = \lceil \log_2(1/P(x)) \rceil + 1$ bits, where $P(x)$ is the probability of the sequence ($= u - l$). This gives a prefix code with average length within $2/m$ bits per symbol of the entropy for a sequence of length m.

- **Practical implementation:** integer arithmetic with rescaling: when the interval lies in [0, 0.5), send 0 and double (E1); when in [0.5, 1), send 1 and map 2x − 1 (E2); when it straddles the middle in [0.25, 0.75), use E3 scaling and remember to send the opposite bit later.
- **Decoding:** find the interval containing the tag, output that symbol, update l and u, and repeat (the length or an end-of-sequence symbol must be known).

Arithmetic vs Huffman

Huffman	Arithmetic
Codes each symbol separately with an integer number of bits	Codes the whole sequence; effectively fractional bits per symbol
Average length within 1 bit of entropy per symbol	Approaches entropy for long sequences
Inefficient for highly skewed probabilities (for example P = 0.95)	Very efficient for skewed sources and small alphabets (binary)
Adaptive version is complex (tree updates)	Easily adaptive: just update counts
Simple and fast	More computation; was patent-restricted in the past

JBIG and JBIG2: for bi-level images, each pixel is coded with an adaptive binary arithmetic coder (QM coder) using a context of neighbouring pixels (10 pixels in a three-line or two-line template). JBIG supports progressive transmission (resolution reduction). JBIG2 (2000) segments a page into text (symbols coded by pattern matching and substitution), halftone and generic regions; it can be lossless or lossy.

Dictionary techniques

- **Static dictionary (digram coding):** the dictionary holds all single letters plus the most frequent pairs (digrams such as "th", "e ", "in"); a pair found in the dictionary is coded with one index.
- **LZ77 (Ziv and Lempel, 1977):** window = search buffer (recently coded text) + look-ahead buffer. Find the longest match of the look-ahead in the search buffer; output ⟨offset, length, next symbol⟩. Matches may overlap into the look-ahead buffer. Used in ZIP, gzip, PNG (DEFLATE = LZ77 + Huffman).
- **LZ78 (1978):** no window; a dictionary of phrases; output ⟨index of longest matching phrase, next symbol⟩ and add phrase + symbol to the dictionary.
- **LZW (Welch, 1984):** dictionary starts with all single symbols; read until w + c is not in the dictionary; output index of w; add w + c; continue with c. The decoder rebuilds the same dictionary (special case: a code not yet in the dictionary means w + first symbol of w).

Applications of LZW and related methods

- **UNIX compress:** LZW with codes starting at 9 bits and growing to 16 bits; when the dictionary is full and the ratio drops, it is reset.
- **GIF (Graphics Interchange Format):** LZW on the colour indices (palette of up to 256 colours), with variable code size and clear codes; lossless for palette images.
- **V.42 bis:** ITU-T modem standard using an LZW variant with a limited dictionary (512 to 2,048 entries), deleting old entries, and a transparent mode when data is not compressible.

PPM (Prediction by Partial Matching)

- Keeps statistics of which symbols followed each context of length 0 to k.

- To code a symbol, start with the longest context (order k). If the symbol has occurred there, code it with arithmetic coding using that context's counts. If not, send an **escape** symbol and drop to a shorter context; at order −1 all symbols are equally likely.
- **Escape probability** methods: PPMA (count 1), PPMB, PPMC (escape count = number of distinct symbols seen in the context).
- **Exclusion principle:** when falling back to a shorter context, exclude symbols that were already present in the longer context (they cannot be the current symbol), which raises the probability of the remaining ones.
- Context length: longer contexts predict better but occur rarely; PPM* uses unbounded context.

Burrows-Wheeler Transform: form all cyclic rotations of the block, sort them, output the last column L and the row index of the original string. The decoder rebuilds the sorted first column F (sort L) and follows the L→F mapping. BWT does not compress itself; it groups symbols so that MTF + run-length + Huffman/arithmetic coding compresses well (bzip2).

Move-to-front: keep a list of the alphabet; output each symbol's index and move it to the front. After BWT, the output has many zeros and small numbers.

CALIC (Context Adaptive Lossless Image Compression): uses gradients in the horizontal and vertical directions to choose a prediction (gradient-adjusted prediction), classifies contexts by texture and error energy, corrects the prediction by the average error in each context (bias cancellation), and codes the residual with context-based arithmetic coding.

JPEG-LS (LOCO-I): median edge detector predictor: for neighbours a (left), b (above), c (above-left): if $c \geq \max(a, b)$, prediction = $\min(a, b)$; if $c \leq \min(a, b)$, prediction = $\max(a, b)$; else prediction = $a + b - c$. Contexts from local gradients, bias correction, and Golomb-Rice coding of residuals; a run mode for flat regions; also a near-lossless mode.

Multiresolution approaches: code a low-resolution version first, then add detail (progressive transmission): pyramid coding, HINT (hierarchical interpolation), resolution reduction in JBIG. Lets the user see an approximate image quickly.

Facsimile (fax) encoding: ITU-T Group 3 (T.4): one-dimensional modified Huffman coding of run lengths of white and black pixels on each line (terminating codes for 0 to 63, make-up codes for multiples of 64); two-dimensional modified READ coding relative to the previous line. Group 4 (T.6): MMR (fully two-dimensional, no end-of-line codes, for error-free networks). JBIG and JBIG2 offer better compression.

Dynamic Markov compression (DMC, Cormack and Horspool): a bit-level adaptive method; it builds a finite-state Markov model, starting small and cloning states when they are visited often, and predicts each bit's probability for binary arithmetic coding.

🧮 Solved examples

Example 1 (arithmetic coding). Symbols a1 = 0.8, a2 = 0.02, a3 = 0.18, so F(a1) = 0.8, F(a2) = 0.82, F(a3) = 1. Encode the sequence a1 a3 a2 a1.

Symbol	Lower l	Upper u
start	0	1
a1	0 + 1 × 0 = 0	0 + 1 × 0.8 = 0.8

Symbol	Lower l	Upper u
a3	0 + 0.8 × 0.82 = 0.656	0 + 0.8 × 1 = 0.8
a2	0.656 + 0.144 × 0.8 = 0.7712	0.656 + 0.144 × 0.82 = 0.77408
a1	0.7712 + 0.00288 × 0 = 0.7712	0.7712 + 0.00288 × 0.8 = 0.773504

Tag = (0.7712 + 0.773504)/2 = **0.772352**. Sequence probability = 0.773504 − 0.7712 = 0.002304, so the binary code needs ⌈log2(1/0.002304)⌉ + 1 = **10 bits** (versus 4 × 2 = 8 bits for a fixed-length code with these 3 symbols; arithmetic coding gains on long sequences, not on 4 symbols).

Example 2 (LZ77). Encode "cabracadabrarrarrad" with a search buffer of 7 symbols and a look-ahead of 6:

Step	Output ⟨offset, length, next⟩	Coded so far
1	⟨0, 0, c⟩	c
2	⟨0, 0, a⟩	ca
3	⟨0, 0, b⟩	cab
4	⟨0, 0, r⟩	cabr
5	⟨3, 1, c⟩	cabrac
6	⟨2, 1, d⟩	cabracad
7	⟨7, 4, r⟩	cabracadabrar
8	⟨3, 5, d⟩	cabracadabrarrarrad

Step 8 copies 5 symbols starting 3 positions back, while those symbols are still being written (an overlapping match), giving "rarra", and then adds d.

Example 3 (LZ78). Encode "ABBCBCABABCAABCAAB":

Output	Dictionary entry
⟨0, A⟩	1: A
⟨0, B⟩	2: B
⟨2, C⟩	3: BC
⟨3, A⟩	4: BCA
⟨2, A⟩	5: BA
⟨4, A⟩	6: BCAA
⟨6, B⟩	7: BCAAB

Example 4 (LZW). Initial dictionary A = 1, B = 2, C = 3. Encode "ABABABA":

- w = A, next B: AB not in dictionary → output 1, add AB = 4, w = B.
- next A: BA not in → output 2, add BA = 5, w = A.
- next B: AB is 4 → w = AB; next A: ABA not in → output 4, add ABA = 6, w = A.

- next B: AB; next A: ABA = 6 → w = ABA; end → output 6.
- Output: **1, 2, 4, 6** (4 codes for 7 symbols). The decoder meets code 6 before it is fully defined and handles it with the special case (w + first symbol of w = AB + A = ABA).

Example 5 (BWT and MTF). BWT of "banana$" ($ is the end marker, sorted first):

- Sorted rotations: $banana, a$banan, ana$ban, anana$b, banana$, na$bana, nana$ba.
- Last column L = **annb$aa**; the original string is row **4** (counting from 0).
- MTF with initial list ($, a, b, n): a → 1; n → 3; n → 0; b → 3; $ → 3; a → 3; a → 0. Output **1 3 0 3 3 3 0**. On longer texts, MTF output is dominated by zeros and small numbers.

Example 6 (JPEG-LS predictor). a = 100, b = 110, c = 90: c ≤ min(a, b) = 100, so prediction = max(a, b) = **110** (an edge is detected). If a = 100, b = 110, c = 105, then prediction = a + b − c = **105**.

Formulas to remember

- $l(k) = l(k-1) + (u(k-1) - l(k-1))\, F(x_k - 1)$; $u(k) = l(k-1) + (u(k-1) - l(k-1))\, F(x_k)$.
- Tag = (l + u)/2; binary length = $\lceil \log_2(1/P(x)) \rceil + 1$.
- LZ77 triple ⟨o, l, c⟩; LZ78 pair ⟨i, c⟩; LZW sends only indices.
- JPEG-LS MED predictor: min(a, b), max(a, b), or a + b − c.

⭐ Important questions

1. Explain arithmetic coding: coding a sequence, generating the tag and the binary code. Compare it with Huffman coding.
2. Explain JBIG and JBIG2.
3. Explain static and adaptive dictionary techniques. Explain LZ77, LZ78 and LZW with examples.
4. Explain applications of LZW: UNIX compress, GIF and V.42 bis.
5. Explain PPM with escape symbols, context length and the exclusion principle.
6. Explain the Burrows-Wheeler transform and move-to-front coding with an example.
7. Write notes on CALIC, JPEG-LS, multiresolution approaches, facsimile encoding and dynamic Markov compression.

UNIT 4 Distortion Criteria and Scalar Quantization

✍️ 2-mark definitions

- **Distortion:** a measure of the difference between the original x and the reconstruction y.
- **Squared error measure:** $d(x, y) = (x - y)^2$. **Mean squared error (MSE):** $\sigma d^2 = (1/N) \Sigma (xn - yn)^2$.
- **Absolute difference measure:** $d(x, y) = |x - y|$.
- **Signal-to-noise ratio (SNR):** $10 \log10(\sigma x^2/\sigma d^2)$ dB, where σx^2 is the signal variance.
- **Peak SNR (PSNR):** $10 \log10(xpeak^2/\sigma d^2)$ dB; for 8-bit images xpeak = 255.
- **Human visual and auditory system models:** perceptual models that weight distortion by how visible or audible it is (for example frequency sensitivity, masking).
- **Quantization:** mapping a large (possibly continuous) set of input values to a smaller set of output values; the main source of loss in lossy compression.
- **Scalar quantizer:** quantizes each sample separately.
- **Decision boundaries and reconstruction levels:** the input range is divided into intervals by decision boundaries bi; each interval maps to one reconstruction (output) level yi.
- **Uniform quantizer:** all intervals (except possibly the outer ones) have equal width Δ (step size), and reconstruction levels are the midpoints.
- **Midrise vs midtread quantizer:** midrise has no zero output level (even number of levels); midtread has zero as an output level (odd number of levels; good for signals with many zeros).
- **Granular and overload noise:** granular noise is the error inside the quantizer's range; overload noise occurs when the input exceeds the range.
- **Adaptive quantizer:** a quantizer whose step size changes according to the input statistics.
- **Non-uniform quantizer:** a quantizer with smaller steps where the input is more probable (pdf-optimized Lloyd-Max quantizer, or companded quantizer).
- **Companding:** compress the input with a non-linear function, quantize uniformly, then expand at the decoder (μ-law, A-law in telephony).

💡 Core concepts

Distortion criteria: MSE, SNR, PSNR, absolute error; perceptual criteria based on the human visual system (contrast sensitivity, masking, lower sensitivity to high frequencies and to colour than brightness) and human auditory system (frequency range 20 Hz to 20 kHz, threshold of hearing, frequency and temporal masking). Rate-distortion theory: R(D) gives the minimum rate needed to achieve distortion D; for a Gaussian source $R(D) = ½ \log2(\sigma^2/D)$.

Models used in lossy coding: probability models (uniform, Gaussian, Laplacian, gamma), linear system models (AR, MA, ARMA for predictive coding), physical models (speech production).

The quantization problem: choose the number of levels M (rate R = log2 M), decision boundaries bi and reconstruction levels yi to minimize the mean squared quantization error: $\sigma q^2 = \Sigma$ over i of $\int$ from b(i−1) to b(i) of $(x - yi)^2 f(x) dx$.

Uniform quantizer

- For an input uniformly distributed in [−Xmax, Xmax] with M levels: Δ = 2Xmax/M; error is uniform in [−Δ/2, Δ/2], so σ_q^2 = **$\Delta^2/12$**.
- SNR = 10 log10(σ_x^2/σ_q^2) = **6.02n dB** for an n-bit quantizer with a uniform input (each extra bit adds about 6 dB).
- For non-uniform inputs (Gaussian, Laplacian), the step size is chosen to balance granular and overload noise.

Adaptive quantization

- **Forward adaptive (off-line):** the encoder analyses a block of input, chooses the step size, and sends it as side information.
- **Backward adaptive (on-line):** the step size is adapted from past quantized outputs, which the decoder also has, so no side information is needed. **Jayant quantizer:** after each output, multiply Δ by a factor Mk depending on which level was used (expand when outer levels are used, shrink for inner levels).

Non-uniform quantization

- **pdf-optimized (Lloyd-Max) quantizer:** necessary conditions: each reconstruction level is the centroid of its interval, yi = ∫ x f(x) dx / ∫ f(x) dx over the interval; each decision boundary is the midpoint of adjacent reconstruction levels, bi = (yi + yi+1)/2. Solved iteratively.
- **Companded quantization:** μ-law: c(x) = Xmax ln(1 + μ|x|/Xmax)/ln(1 + μ) × sgn(x), with μ = 255 (North America, Japan); A-law with A = 87.6 (Europe, India). Gives roughly constant SNR over a wide range of input levels (good for speech).
- **Entropy-coded quantization:** a uniform quantizer followed by entropy coding of its outputs is nearly optimal at high rates.

🧮 Solved examples

Example 1 (uniform quantizer). Input uniform in [−4, 4], 3-bit quantizer (M = 8).

- Δ = 8/8 = **1**. Decision boundaries: −4, −3, ..., 3, 4. Reconstruction levels: −3.5, −2.5, ..., 3.5 (midrise).
- σ_q^2 = 1/12 = **0.0833**; σ_x^2 = $8^2/12$ = 5.333; SNR = 10 log10(5.333/0.0833) = 10 log10 64 = **18.06 dB** (= 6.02 × 3).
- Input 1.3 → interval [1, 2) → output **1.5**; error 0.2.

Example 2 (MSE and PSNR). Original pixels 52, 55, 61, 66; reconstructed 50, 56, 60, 68. Errors 2, −1, 1, −2; MSE = (4 + 1 + 1 + 4)/4 = **2.5**. PSNR = 10 log10(255^2/2.5) = 10 log10(26010) ≈ **44.15 dB**.

Example 3 (Lloyd-Max step). Current levels y1 = −1, y2 = 1 for a symmetric input: boundary b1 = (−1 + 1)/2 = 0. If the centroid of f(x) on [0, ∞) is 0.8, the new y2 = 0.8 (and y1 = −0.8); repeat until the levels stop changing.

Example 4 (rate-distortion). Gaussian source with σ^2 = 1 and allowed MSE D = 0.25: R(D) = ½ log2(1/0.25) = **1 bit/sample**. Halving D again needs ½ bit more per sample.

Formulas to remember

- MSE = (1/N) Σ $(x - y)^2$; SNR = 10 log10(σ_x^2/σ_d^2); PSNR = 10 log10(255^2/MSE).

NEXT ENGINEER _sheerat_

- Uniform: $\Delta = 2X_{max}/M$; $\sigma_q^2 = \Delta^2/12$; SNR ≈ 6.02n dB.
- Lloyd-Max: yi = centroid; bi = (yi + yi+1)/2.
- μ-law: $c(x) = X_{max} \ln(1 + \mu|x|/X_{max})/\ln(1 + \mu)\ \mathrm{sgn}(x)$.
- Gaussian $R(D) = \frac{1}{2} \log_2(\sigma^2/D)$.

⭐ Important questions

1. Explain distortion criteria: MSE, SNR, PSNR, and perceptual criteria based on human vision and hearing.
2. Explain models used in lossy compression. What is the quantization problem?
3. Explain the uniform quantizer and derive its MSE ($\Delta^2/12$) and the 6 dB per bit rule.
4. Explain forward and backward adaptive quantization (Jayant quantizer).
5. Explain non-uniform quantization: pdf-optimized (Lloyd-Max) and companded quantizers.

NEXT ENGINEER _sheerat_

UNIT 5 Vector Quantization

✍️ 2-mark definitions

- **Vector quantization (VQ):** quantizing a block (vector) of L samples together by mapping it to the nearest of K code vectors in a codebook; only the index is sent.
- **Codebook:** the set of K reconstruction vectors (code vectors, codewords) shared by encoder and decoder.
- **Rate of VQ:** R = (log2 K)/L bits per sample.
- **Voronoi region (encoding region):** the set of input vectors closer to a given code vector than to any other.
- **LBG algorithm (Linde, Buzo, Gray, 1980):** an iterative algorithm, a generalization of Lloyd's algorithm, to design a codebook from training vectors.
- **Empty cell problem:** a code vector receives no training vectors during LBG; it is replaced (for example by splitting the most populated cell).
- **Splitting technique:** initialize LBG by starting with one code vector (the mean) and repeatedly splitting each code vector into two slightly perturbed copies.
- **Tree-structured VQ (TSVQ):** a VQ where the search is done down a tree (binary decisions), reducing encoding complexity from K to about 2 log2 K comparisons, at some loss of performance.
- **Structured VQ:** VQ with a codebook having a regular structure (lattice VQ, pyramid VQ, product codes such as gain-shape and mean-removed VQ) to reduce storage and search cost.
- **Lattice VQ:** code vectors are points of a regular lattice, so no codebook needs to be stored and the nearest point can be found quickly.

💡 Core concepts

Advantages of VQ over scalar quantization

1. Exploits correlation between samples (in images, neighbouring pixels are similar), so fewer bits are needed for the same distortion.
2. Can use fractional bit rates per sample (for example 8-bit index for a 16-pixel block = 0.5 bit/pixel).
3. Even for independent samples, VQ cells can be shaped better (space-filling advantage) than the rectangular cells of scalar quantizers.
4. Decoding is very simple: a table look-up. - **Disadvantages:** encoding complexity (search K vectors of dimension L), codebook storage and design, codebook mismatch with data that differs from the training set, edge and blocking artefacts in images.

LBG algorithm

1. Start with an initial codebook {y1, ..., yK} (random, splitting, or pairwise nearest neighbour), set k = 0, D(0) = ∞.
2. **Partition:** assign each training vector to the nearest code vector (Voronoi regions).
3. Compute the average distortion D(k).

4. If (D(k−1) − D(k))/D(k) < ε, stop.
5. **Update:** replace each code vector by the centroid (mean) of the training vectors in its region; go to step 2. - Converges to a local minimum; the result depends on initialization. - **Initialization by splitting:** start with K = 1 (overall mean); split each vector y into y + ε and y − ε; run LBG; repeat until K code vectors. - **Empty cell handling:** replace the empty code vector with a perturbed copy of the vector having the most training points.

Tree-structured VQ: build a binary tree: at the root, split the training data into 2 clusters (LBG with K = 2); at each child, split again; leaves are the final code vectors. Encoding compares the input with 2 vectors at each level: about 2 log2 K comparisons instead of K (for K = 256: 16 instead of 256). Storage roughly doubles (internal nodes), and performance is slightly worse than full-search VQ. Pruned TSVQ removes branches to trade rate for distortion.

Structured VQ

- **Lattice VQ:** code vectors on a lattice (for example Dn, E8); fast quantization, no stored codebook; suited to uniform sources; used with pyramid VQ for Laplacian sources.
- **Product codes:** split the vector into features coded separately: **gain-shape VQ** (code the norm and the normalized direction separately), **mean-removed VQ** (subtract the block mean, code the mean and the residual pattern), multistage (residual) VQ (quantize, then quantize the error with a second codebook).
- **Classified VQ:** different codebooks for different block classes (edges, smooth, texture).

Applications: image compression (blocks of 4 × 4 pixels), speech coding (CELP uses VQ of excitation; LPC parameters are vector-quantized), video, pattern recognition.

🧮 Solved examples

Example 1 (rate of VQ). Image blocks of 4 × 4 = 16 pixels, codebook of K = 256 code vectors: index = 8 bits; rate = 8/16 = **0.5 bits/pixel**, compared with 8 bits/pixel originally (ratio 16:1, ignoring the codebook).

Example 2 (one LBG run). Training vectors (1, 1), (1, 2), (2, 1), (5, 5), (6, 5), (5, 6); initial codebook y1 = (1, 1), y2 = (5, 5).

- Iteration 1 partition: {(1, 1), (1, 2), (2, 1)} → y1; {(5, 5), (6, 5), (5, 6)} → y2. Distortion (squared error) = 0 + 1 + 1 + 0 + 1 + 1 = 4; average = 4/6 = **0.667**.
- Update centroids: y1 = (4/3, 4/3) ≈ **(1.33, 1.33)**; y2 = (16/3, 16/3) ≈ **(5.33, 5.33)**.
- Iteration 2 partition is the same; average distortion = **0.444**. The next update gives the same centroids, so the algorithm has converged.

Example 3 (encoding with a codebook). Codebook {(0, 0), (2, 2), (4, 0)}; input (3, 1): squared distances 10, 2, 2 → tie between (2, 2) and (4, 0); choose the lower index (2, 2) by convention. Input (1, 0): distances 1, 5, 9 → **(0, 0)**.

Example 4 (TSVQ cost). K = 1,024 code vectors: full search = **1,024** distance computations; TSVQ with a binary tree of depth 10 = 2 × 10 = **20** computations.

Formulas to remember

- Rate R = (log2 K)/L bits/sample.

- Distortion $D = (1/N) \Sigma ||x - Q(x)||^2$.
- LBG: nearest-neighbour partition → centroid update → stop when the relative fall in $D < \varepsilon$.
- TSVQ search ≈ 2 log2 K comparisons.

⭐ Important questions

1. What is vector quantization? Explain its advantages over scalar quantization.
2. Explain the LBG algorithm with initialization (splitting) and the empty cell problem. Work through an example.
3. Explain tree-structured vector quantization and its advantages and disadvantages.
4. Explain structured vector quantizers: lattice, gain-shape, mean-removed and multistage VQ.

Last-hour checklist

- ☐ Lossless vs lossy; ratio, rate, distortion; modeling and coding example.
- ☐ Self-information, entropy (1.75 example), source coding theorem.
- ☐ Physical, probability, Markov, composite models.
- ☐ Unique decodability test ({0, 01, 11} yes, {0, 01, 10} no); Kraft inequality; prefix codes.
- ☐ Huffman and minimum variance Huffman (2.2 bits, variance 1.36 vs 0.16, 96.5%).
- ☐ Adaptive Huffman (NYT, sibling property, update).
- ☐ Golomb (m = 5 table), Rice (k = 2: 9 → 11001), Tunstall (7 entries).
- ☐ Arithmetic coding table (tag 0.772352, 10 bits); arithmetic vs Huffman; JBIG, JBIG2.
- ☐ LZ77 table, LZ78 table, LZW (1, 2, 4, 6); compress, GIF, V.42 bis.
- ☐ PPM (escape, exclusion); BWT (annb$aa, index 4) and MTF (1 3 0 3 3 3 0); CALIC; JPEG-LS MED; fax; DMC.
- ☐ Distortion measures; uniform quantizer ($\Delta^2/12$, 6.02n dB, 18.06 dB example); adaptive (Jayant); Lloyd-Max; companding.
- ☐ VQ advantages; LBG example (0.667 → 0.444); TSVQ; structured VQ.

Books prescribed by AKTU

These are the books named in the official AKTU syllabus. Buy or borrow them through a bookshop or your college library. The WorldCat links show which libraries hold a copy.

#	Title	Author(s)	Publisher	Where to get it legally
1	Introduction to Data Compression	Khalid Sayood	Morgan Kaufmann	WorldCat
2	Elements of Data Compression	Adam Drozdek	Cengage Learning	WorldCat

#	Title	Author(s)	Publisher	Where to get it legally
3	Data Compression: The Complete Reference (4th ed.)	David Salomon	Springer	WorldCat
4	Text Compression	Timothy C. Bell, John G. Cleary, Ian H. Witten	Prentice Hall	WorldCat

Syllabus source: AKTU B.Tech 3rd year CS/IT syllabus, 2024-25 (copy hosted by an affiliated college, PDF)

NEXT ENGINEER
sheerat